Exercício 1

Contexto

Você está criando um sistema simples que precisa **notificar** o usuário quando uma conta é criada.

Objetivo

Aplicar **Inversão de Controle** removendo a responsabilidade da classe principal de **instanciar** sua dependência.

Requisitos obrigatórios

1. Crie uma classe `ContaService` que possua o método:

- `CriarConta(string nome, string email)`

2. `ContaService` deve notificar o usuário com a mensagem:

- `"Conta criada para <nome> (<email>)"`

3. A notificação deve ser feita por meio de um contrato (interface):

- `INotificador` com método `Enviar(string mensagem)`

4. Implemente 2 notificadores:

- `NotificadorEmail` (imprime no console: `EMAIL: <mensagem>`)
- `NotificadorSms` (imprime no console: `SMS: <mensagem>`)

5. **Regra IoC (obrigatória):**

- `ContaService` **não pode** fazer `new NotificadorEmail()` nem `new NotificadorSms()` dentro dela.
- `ContaService` deve receber `INotificador` via **construtor**.

6. No `Program.cs`, demonstre:

- Uma execução usando `NotificadorEmail`
- Outra execução usando `NotificadorSms`

Critérios de aceite

- Trocar Email por SMS deve exigir alteração **somente no Program.cs**
- `ContaService` deve depender de `INotificador` (não de classe concreta)

Exercício 2

Contexto

Você vai criar um mini fluxo de **processamento de pedidos** com:

- cálculo de total
- aplicação de desconto
- pagamento
- logging

Tudo **desacoplado** e configurado via **container de DI** (IoC).

Objetivo

Aplicar **IoC/DI no padrão do mercado** (Microsoft.Extensions.DependencyInjection) e estruturar dependências corretamente.

Requisitos obrigatórios

(A) Domínio do Pedido

1. Crie as classes:

- `Produto` (Nome, Preco)
- `ItemPedido` (Produto, Quantidade, Subtotal())
- `Pedido` (lista interna de itens + AdicionarItem + TotalBruto)

Regras:

- `Preco > 0`
- `Quantidade >= 1`
- `Pedido` não expõe lista mutável.

(B) Desconto (polimorfismo)

2. Crie a interface:

```
public interface IDesconto
{
    string Nome { get; }
    decimal CalcularDesconto(Pedido pedido);
}
```

Implemente 2 descontos:

- `DescontoPercentual( decimal percentual )` → ex.: 0.10m = 10%
- `DescontoPorQuantidade( int minItens, decimal valorFixo )` → se total de itens >= minItens, aplica valor fixo

Regra: o processador deve aplicar **todos** os descontos registrados via DI, somando-os.

(C) Pagamento (polimorfismo)

3. Crie a interface:

```
public interface IPagamento
{
    string Nome { get; }
    void Pagar(decimal valor);
}
```

Implemente:

- `PagamentoPix`
- `PagamentoCartao`

(D) Logger

4. Crie `ILogger` com `Info` e `Error` e implemente `ConsoleLogger`.

(E) Processador

5. Crie `PedidoProcessor` que recebe via construtor:

- `IEnumerable<IDesconto> descontos`
- `IPagamento pagamento`
- `ILogger logger`

E possui método:

- `Processar(Pedido pedido)`

O método deve:

1. calcular bruto
2. calcular total de descontos (somando)
3. calcular total final (bruto - descontos; nunca pode ficar < 0)
4. pagar
5. logar o resumo

(F) DI/IoC obrigatório

6. No `Program.cs` você deve:

- configurar `ServiceCollection`
- registrar tudo
- resolver o `PedidoProcessor` pelo container
- executar o cenário

Regras:

- `PedidoProcessor` não pode instanciar `new PagamentoPix()`, `new Desconto...()` etc.
- `PedidoProcessor` não pode ter `if/switch` para escolher pagamento ou descontos
- Trocar PIX por Cartão deve ser feito **somente no registro do DI** (Program)